

针对《软件开发框架II》的复习知识点ppt的讲解
PPT 主题是 **SSM 核心知识复习**，也就是：

Spring + Spring MVC + MyBatis

它主要复习 Java 后端开发里非常经典的一套技术组合：
Spring 负责管理对象和业务逻辑，Spring MVC 负责处理网页/API 请求，MyBatis 负责操作数据库。
PPT 分成 Spring Core、Spring AOP、MyBatis、Spring MVC、综合应用五大部分。

一、先搞懂 SSM 是什么

SSM 是三个框架的组合：

名称	主要作用	通俗理解
Spring	管理对象、管理业务、管理事务	后台“大管家”
Spring MVC	接收浏览器/前端请求，返回页面或 JSON	门口“接待员”
MyBatis	操作数据库，执行 SQL	数据库“翻译官”

Table 1

比如你做一个“学生管理系统”：

- 用户点击“查询学生”
- Spring MVC 接收请求
 - 调用 Service 业务层
 - Service 调用 MyBatis 查询数据库
 - 数据库返回学生数据
 - Spring MVC 把结果返回给前端。

这就是 SSM 的典型工作流程。

二、Spring Core: Spring 的核心

1. 1. IoC: 控制反转

PPT 里第一个重点是 **IoC**，英文是：

Inversion of Control, 控制反转。

你先记一句话：

IoC 就是：对象不用你自己 new，而是交给 Spring 容器来创建和管理。

以前我们写代码可能这样：

```
1 UserDao userDao = new UserDaoImpl();
2 UserService userService = new UserServiceImpl(userDao);
```

Fence 1

这表示程序员自己创建对象。

用了 Spring 后，你不直接 new，而是告诉 Spring：

这些类你帮我创建，需要的时候你帮我拿来用。

Spring 就像一个“大仓库”或者“外卖平台”。

你想吃披萨，不用自己买面粉、揉面、烤披萨。

你只需要点外卖，平台帮你安排。

同理：

你想用 `UserService`，不用自己 new。

Spring 容器帮你创建好，然后注入给你。

2. 2. DI: 依赖注入

DI 是：

Dependency Injection, 依赖注入。

它和 IoC 经常放在一起讲。

IoC 说的是：

对象由 Spring 创建。

DI 说的是：

对象需要的其他对象，也由 Spring 自动塞进去。

比如：

```
1 public class UserService {
2     private UserDao userDao;
3 }
```

Fence 2

`UserService` 要依赖 `UserDao` 。
没有 `UserDao` ，它就没法查数据库。

以前你要自己写：

```
1 | userService.setUserDao(new UserDaoImpl());
```

Fence 3

现在 Spring 可以自动帮你注入。

3.3. Bean 是什么

PPT 多次提到 **Bean**。

你可以先简单理解：

被 Spring 容器管理的对象，就叫 Bean。

比如这些类：

```
1 | @Service
2 | public class UserServiceImpl {
3 | }
4 | @Repository
5 | public class UserDaoImpl {
6 | }
7 | @Controller
8 | public class UserController {
9 | }
```

Fence 4

只要它们被 Spring 扫描并创建出来，它们就是 Bean。

Bean 不是一种特殊的 Java 类，它本质上就是普通对象，只不过由 Spring 管理。

4.4. Spring IoC 容器负责什么

PPT 里讲了 IoC 容器的四个职责：

4.1 第一，创建对象

比如创建：

```
1 | UserServiceImpl userService = new UserServiceImpl();
```

Fence 5

这个过程交给 Spring 完成。

4.2 第二，依赖注入

比如 `UserService` 需要 `UserDao` ，Spring 自动注入。

```
1  @Service
2  public class UserServiceImpl {
3      @Autowired
4      private UserDao userDao;
5  }
```

Fence 6

这里的 `@Autowired` 表示：

Spring，请你帮我把合适的 `UserDao` 对象注入进来。

4.3 第三，生命周期管理

对象从创建到销毁，有完整过程：

1. 创建对象
2. 注入属性
3. 初始化
4. 使用
5. 销毁

Spring 可以控制这些阶段。

比如数据库连接、线程池、缓存等资源，需要初始化和销毁，Spring 可以统一管理。

4.4 第四，配置管理

Spring 可以统一管理配置。

比如数据库地址：

```
1  jdbc.url=jdbc:mysql://localhost:3306/mydb
2  jdbc.user=root
3  jdbc.password=123456
```

Fence 7

这些配置可以放在外部文件里，不写死在 Java 代码中。

这样以后换数据库地址，不用改 Java 源码，只改配置文件即可。

三、依赖注入的两种方式

PPT 重点讲了两种：

1. Setter 注入
2. 构造函数注入

1. 1. Setter 注入

示例：

```
1  @Service
2  public class UserServiceImpl {
3      private UserDao userDao;
4
5      @Autowired
6      public void setUserDao(UserDao userDao) {
7          this.userDao = userDao;
8      }
9  }
```

Fence 8

意思是：

Spring 创建 `ServiceImpl` 后，再调用 `setUserDao()` 方法，把 `UserDao` 传进去。

优点：

灵活，可以后期修改依赖。

缺点：

对象刚创建出来时，可能还没有依赖。

如果在注入前就使用，可能出现空指针异常。

2. 2. 构造函数注入

示例：

```
1  @Service
2  public class UserServiceImpl {
3      private final UserDao userDao;
4
5      @Autowired
6      public UserServiceImpl(UserDao userDao) {
7          this.userDao = userDao;
8      }
9  }
```

Fence 9

意思是：

创建 `ServiceImpl` 的时候，必须传入 `UserDao`。

优点：

对象一创建就是完整的。

不会出现“对象创建了，但依赖还没注入”的问题。

PPT 里也强调：

构造函数注入更安全，是推荐方式。

你可以记：

■ 必须依赖，用构造函数注入；可选依赖，可以用 Setter 注入。

四、Bean 作用域和组件扫描

1. 1. Singleton 单例

PPT 说 Spring Bean 默认作用域是：

Singleton，单例。

意思是：

一个 Bean 在 Spring 容器里通常只有一个对象。

比如：

```
1 UserService userService1 = context.getBean(UserService.class);
2 UserService userService2 = context.getBean(UserService.class);
```

Fence 10

这两个拿到的其实是同一个对象。

通俗理解：

一个班级只有一个班主任对象，大家都找同一个班主任。

2. 2. 单例适合什么场景

适合：

- Service
- DAO
- 工具类
- Controller

因为它们通常不保存每个用户自己的数据。

3. 3. 单例要注意线程安全

如果一个单例对象里面保存了可变数据，就可能出问题。

例如：

```
1 @Service
2 public class UserService {
3     private String username;
4 }
```

Fence 11

如果多个用户同时访问这个 Service，大家都改 `username`，就可能互相影响。

所以一般 Service 里面不要保存用户状态。

4. 4. 组件扫描是什么

PPT 讲了组件扫描：

```
1 | <context:component-scan base-package="com.example"/>
```

Fence 12

或者 Java 配置：

```
1 | @ComponentScan(basePackages = "com.example")
```

Fence 13

意思是：

Spring 去指定包下面扫描类，看看哪些类上有注解，如果有，就把它们创建成 Bean。

常见注解：

注解	用在哪一层	作用
@Repository	DAO 层	数据访问层，操作数据库
@Service	Service 层	业务逻辑层
@Controller	Controller 层	控制层，处理网页请求
@Component	通用组件	不好归类时使用

Table 2

五、Spring AOP：面向切面编程

现在进入第二章。

AOP 是：

Aspect-Oriented Programming，面向切面编程。

你先记住：

AOP 用来处理很多地方都需要的“公共功能”。

比如：

- 日志记录
- 权限校验
- 事务管理
- 性能统计
- 异常处理

这些功能不是核心业务，但很多地方都要用。

1. 1. 为什么需要 AOP

假设你有很多方法：

```
1  addUser()  
2  deleteUser()  
3  updateUser()  
4  queryUser()
```

Fence 14

每个方法都要记录日志。

如果不用 AOP，你可能每个方法都写：

```
1  System.out.println("开始执行");  
2  System.out.println("执行结束");
```

Fence 15

代码就会重复。

AOP 的思想是：

把这些公共代码单独抽出来，在方法执行前后自动插进去。

这样业务代码只写业务。

六、AOP 的代理机制

PPT 讲了两种底层代理：

1. JDK 动态代理
 2. CGLIB 代理
-

1. 1. 代理是什么

代理可以理解为“中间人”。

比如你要找明星拍广告，你一般不会直接找明星本人，而是找经纪人。
经纪人可以在中间加流程，比如谈价格、签合同、安排时间。

在 Spring AOP 中，代理对象也是类似：

你以为你调用的是原对象，实际上先经过代理对象。
代理对象可以在方法前后加功能。

2. 2. JDK 动态代理

特点：

要求目标类必须实现接口。

例如：

```
1 public interface UserService {
2     void addUser();
3 }
4 public class UserServiceImpl implements UserService {
5     public void addUser() {
6         System.out.println("添加用户");
7     }
8 }
```

Fence 16

如果有接口，Spring 默认可以使用 JDK 动态代理。

优点：

- Java 原生支持
- 不需要额外依赖
- 适合面向接口编程

限制：

- 必须有接口
-

3. 3. CGLIB 代理

如果类没有实现接口，Spring 可以用 CGLIB。

CGLIB 的方式是：

生成目标类的子类，通过继承来增强方法。

例如原来有：

```
1 public class UserService {  
2     public void addUser() {}  
3 }
```

Fence 17

CGLIB 会创建一个类似：

```
1 public class UserServiceProxy extends UserService {  
2 }
```

Fence 18

限制：

- 目标类不能是 `final`
- 方法不能是 `final`
- 私有方法不能被代理

因为 `final` 类不能被继承，`final` 方法不能被重写。

七、AOP 的通知类型

PPT 讲了五种 Advice，也就是通知。

通知就是：

切面代码在什么时候执行。

1. 1. @Before 前置通知

目标方法执行前运行。

适合：

- 权限校验
- 参数检查
- 打印开始日志

例子：

```
1 @Before("execution(* com.example.service.*.*(..))")
2 public void before() {
3     System.out.println("方法执行前");
4 }
```

Fence 19

2. 2. @AfterReturning 后置通知

目标方法正常执行完后运行。

注意：只有没有异常，才会执行。

适合：

- 获取返回值
 - 成功日志
 - 缓存更新
-

3. 3. @Around 环绕通知

最强大。

它可以包住整个方法：

- 方法前做事
- 决定是否执行原方法
- 方法后做事
- 捕获异常
- 修改返回值

通俗理解：

`@Around` 像完整包围目标方法的一层壳。

4. 4. `@AfterThrowing` 异常通知

只有目标方法抛异常时才执行。

适合：

- 错误日志
 - 异常告警
 - 事务回滚辅助
-

5. 5. `@After` 最终通知

无论方法成功还是失败，都会执行。

类似 Java 里的：

```
1  finally {  
2      // 一定执行  
3  }
```

Fence 20

适合：

- 释放资源
 - 清理临时变量
 - 关闭连接
-

八、声明式事务：@Transactional

这一块很重要。

事务是什么？

事务就是一组数据库操作，要么全部成功，要么全部失败。

经典例子：转账。

A 给 B 转 100 元：

1. A 余额减少 100
2. B 余额增加 100

这两个操作必须一起成功。

如果 A 扣钱成功，B 加钱失败，就出大问题。

所以要事务。

1.1. @Transactional 是什么

PPT 说：

@Transactional 基于 Spring AOP 动态代理实现。

你可以理解为：

Spring 在方法执行前自动开启事务，方法成功后提交事务，方法出错后回滚事务。

例如：

```
1 | @Transactional
2 | public void transfer() {
3 |     accountDao.minusMoney();
4 |     accountDao.addMoney();
5 | }
```

Fence 21

你不需要手动写：

```
1 | connection.commit();
2 | connection.rollback();
```

Fence 22

Spring 帮你做。

2.2. 事务传播行为

PPT 重点讲了两个：

2.1 REQUIRED

默认值。

意思是：

如果当前有事务，就加入当前事务；如果没有事务，就新建一个事务。

这是最常用的。

比如：

```
1  @Transactional
2  public void methodA() {
3      methodB();
4  }
```

Fence 23

如果 `methodB()` 也是 REQUIRED，它会加入 `methodA()` 的事务。

2.2 REQUIRES_NEW

意思是：

不管当前有没有事务，都新开一个事务。

如果原来有事务，先挂起原事务。

常见场景：

比如主业务失败了，但日志必须保存。

```
1  @Transactional
2  public void buy() {
3      orderService.createOrder();
4      logService.saveLog(); // 可以用 REQUIRES_NEW
5      throw new RuntimeException();
6  }
```

Fence 24

如果日志方法用 `REQUIRES_NEW`，即使订单事务回滚，日志事务也可以单独提交。

3.3. 默认什么异常会回滚

PPT 里说得很关键：

Spring 默认只对：

- `RuntimeException`
- `Error`

进行回滚。

比如：

```
1 | throw new RuntimeException();
```

Fence 25

会回滚。

但是像：

```
1 | throw new IOException();  
2 | throw new SQLException();
```

Fence 26

这些受检异常默认不一定回滚。

如果你想让这些异常也回滚，要写：

```
1 | @Transactional(rollbackFor = Exception.class)
```

Fence 27

这句话考试和面试都很常见。

九、MyBatis：操作数据库的框架

MyBatis 是持久层框架。

持久层就是：

和数据库打交道的那一层。

比如：

- 查询用户
- 添加订单
- 修改商品
- 删除评论

这些都属于数据库操作。

十、MyBatis 的 `#{}` 和 `${}`

PPT 重点讲了：

1. `#{}`
2. `${}`

这是 MyBatis 里非常重要、非常容易考的点。

1. 1. `#{}` ：安全，推荐使用

例子：

```
1 | SELECT * FROM user WHERE id = #{userId}
```

Fence 28

MyBatis 底层会把它变成：

```
1 | SELECT * FROM user WHERE id = ?
```

Fence 29

然后把参数传进去。

这种方式叫：

预编译。

优点：

- 安全
- 防止 SQL 注入
- 自动处理类型转换

日常开发优先用 `#{}` 。

2. 2. `${}` ：字符串拼接，有风险

例子：

```
1 | SELECT * FROM goods ORDER BY ${sortField}
```

Fence 30

`${}` 会直接把内容拼接进 SQL。

比如传入：

```
1 | price
```

Fence 31

最终 SQL 是：

```
1 | SELECT * FROM goods ORDER BY price
```

Fence 32

看起来没问题。

但是如果用户恶意传入：

```
1 | price; DROP TABLE goods;
```

Fence 33

就可能造成 SQL 注入。

所以 `${}` 很危险。

3.3. 什么时候必须用 `${}`

有些地方不能用 `#{}` ，比如：

- 表名
- 字段名
- 排序字段

例如：

```
1 | ORDER BY #{sortField}
```

Fence 34

这样通常不行，因为字段名不能作为普通参数预编译。

这时可以用 `${}`，但必须做白名单校验。

比如只允许：

```
1 | name
2 | price
3 | create_time
```

Fence 35

其他一律拒绝。

你可以记：

传普通值用 `#{}` ；拼 SQL 结构才考虑 `${}`，但必须小心。

十一、MyBatis 动态 SQL

动态 SQL 是 MyBatis 很重要的功能。

它解决的问题是：

根据不同条件，动态拼接 SQL。

比如查询用户：

用户可能输入姓名，也可能输入年龄，也可能两个都不输入。

你不能写死 SQL。

1.1. <if>

根据条件决定是否拼接某段 SQL。

例子：

```
1 <select id="findUsers" resultType="User">
2     SELECT * FROM user
3     WHERE 1=1
4     <if test="name != null">
5         AND name = #{name}
6     </if>
7 </select>
```

Fence 36

如果 name 不为空，就加上姓名条件。

如果 name 为空，就不加。

2.2. <where>

<where> 可以自动处理 WHERE 和多余的 AND/OR。

例子：

```
1 <select id="findUsers" resultType="User">
2     SELECT * FROM user
3     <where>
4         <if test="name != null">
5             AND name = #{name}
6         </if>
7         <if test="age != null">
8             AND age = #{age}
9         </if>
10    </where>
11 </select>
```

Fence 37

如果 name 和 age 都为空，MyBatis 不会生成 WHERE。

如果有条件，它会自动加 WHERE，并去掉前面多余的 AND。

非常好用。

3.3. <set>

专门用于 update。

例子：

```
1  <update id="updateUser">
2      UPDATE user
3      <set>
4          <if test="name != null">
5              name = #{name},
6          </if>
7          <if test="age != null">
8              age = #{age},
9          </if>
10     </set>
11     WHERE id = #{id}
12 </update>
```

Fence 38

<set> 会自动去掉最后多余的逗号。

4.4. <foreach>

用于遍历集合。

常用于：

```
1  WHERE id IN (1, 2, 3)
```

Fence 39

MyBatis 写法：

```
1  <select id="findByIds" resultType="User">
2      SELECT * FROM user
3      WHERE id IN
4          <foreach collection="ids" item="id" open="(" separator="," close=")">
5              #{id}
6          </foreach>
7  </select>
```

Fence 40

如果 ids 是 [1,2,3]，生成：

```
1  WHERE id IN (1,2,3)
```

Fence 41

5. 5. <choose>

类似 Java 的 switch。

只会选择一个条件执行。

```
1  <choose>
2    <when test="name != null">
3      name = #{name}
4    </when>
5    <when test="age != null">
6      age = #{age}
7    </when>
8    <otherwise>
9      1 = 1
10   </otherwise>
11 </choose>
```

Fence 42

十二、MyBatis 缓存机制

PPT 讲了一级缓存和二级缓存。

缓存就是：

把查过的数据临时存起来，下次不用再查数据库。

1. 1. 一级缓存

一级缓存是：

SqlSession 级别。

特点：

- 默认开启
- 同一个 SqlSession 内有效
- 执行增删改会清空
- SqlSession 关闭后失效

比如同一个 SqlSession 中：

```
1 userMapper.findById(1);
2 userMapper.findById(1);
```

Fence 43

第二次可能直接从缓存拿，不查数据库。

2. 2. 二级缓存

二级缓存是：

Mapper 级别。

特点：

- 默认不一定开启
- 需要手动配置
- 可以跨 SqlSession 共享
- 适合查询多、修改少的数据

比如字典表、分类表、地区表等数据比较稳定，可以考虑二级缓存。

但在真实项目中，二级缓存要谨慎使用，因为数据一致性比较麻烦。

十三、Spring Java Config 配置类

PPT 讲了 Spring 的 Java 配置。

以前 Spring 常用 XML 配置：

```
1 <bean id="userService" class="com.example.UserService"/>
```

Fence 44

现在更常用 Java 配置类。

1. 1. @Configuration

表示当前类是配置类。

```
1 @Configuration
2 public class AppConfig {
3 }
```

Fence 45

它相当于 XML 里的 `<beans>`。

2. 2. @Bean

用于手动创建 Bean。

```
1 @Bean
2 public DataSource dataSource() {
3     return new DriverManagerDataSource();
4 }
```

Fence 46

方法返回的对象会交给 Spring 管理。

方法名默认就是 Bean 的名字。

3. 3. @ComponentScan

开启组件扫描。

```
1 @ComponentScan(basePackages = "com.example")
```

Fence 47

表示扫描 `com.example` 包下面的类。

4. 4. @PropertySource

加载外部配置文件。


```
1 | @PropertySource("classpath:db.properties")
```

Fence 48

比如加载数据库配置:

```
1 | jdbc.url=jdbc:mysql://localhost:3306/mydb  
2 | jdbc.user=root
```

Fence 49

十四、MyBatis 配置文件

PPT 讲了 `mybatis-config.xml`。

它是 MyBatis 的核心配置文件。

主要有三个部分：

1. 1. `<environments>`

配置数据库环境。

```
1  <environments default="dev">
2      <environment id="dev">
3          <transactionManager type="JDBC"/>
4          <dataSource type="POOLED">
5              <property name="driver" value="com.mysql.cj.jdbc.Driver"/>
6              <property name="url" value="jdbc:mysql://localhost:3306/mydb"/>
7          </dataSource>
8      </environment>
9  </environments>
```

Fence 50

它告诉 MyBatis：

- 用什么数据库驱动
- 连接哪个数据库
- 用户名密码是什么
- 用什么事务管理方式

2. 2. `<mappers>`

注册 SQL 映射文件。

```
1  <mappers>
2      <mapper resource="com/example/mapper/UserMapper.xml"/>
3  </mappers>
```

Fence 51

意思是：

MyBatis 去这个 XML 文件里找 SQL。

3. 3. `<settings>`

配置 MyBatis 的全局行为。

比如：

- 是否开启二级缓存

- 是否开启懒加载
 - 使用什么日志实现
-

十五、Spring MVC：Web 请求处理框架

Spring MVC 负责处理浏览器或前端发来的 HTTP 请求。

比如：

```
1  GET /api/employees
2  POST /api/employees
3  DELETE /api/employees/1
```

Fence 52

Spring MVC 要负责：

1. 接收请求
 2. 找到对应方法
 3. 调用业务逻辑
 4. 返回结果
-

十六、@RestController

PPT 重点讲了这个注解。

```
1 @RestController
2 @RequestMapping("/api/employees")
3 public class EmployeeController {
4 }
```

Fence 53

@RestController 等价于：

```
1 @Controller + @ResponseBody
```

Fence 54

意思是：

这个类是控制器，并且方法返回的数据直接写入响应体，通常转换成 JSON。

普通 @Controller 常用于返回页面。

@RestController 常用于返回数据。

1.1. @GetMapping

处理 GET 请求。

通常用于查询。

```
1 @GetMapping
2 public List<Employee> list() {
3     return service.findAll();
4 }
```

Fence 55

对应：

```
1 GET /api/employees
```

Fence 56

2.2. @PostMapping

处理 POST 请求。

通常用于新增。

```
1 @PostMapping
2 public Employee create(@RequestBody Employee emp) {
3     return service.save(emp);
4 }
```

Fence 57

对应：

```
1 | POST /api/employees
```

Fence 58

3.3. @PathVariable

从路径中取参数。

```
1 | @GetMapping("/{id}")
2 | public Employee getById(@PathVariable Integer id) {
3 |     return service.findById(id);
4 | }
```

Fence 59

请求：

```
1 | GET /api/employees/5
```

Fence 60

这里的 5 会绑定到 id。

4.4. @RequestParam

从查询参数中取值。

```
1 | @GetMapping("/search")
2 | public List<Employee> search(@RequestParam String name) {
3 |     return service.findByName(name);
4 | }
```

Fence 61

请求：

```
1 | GET /api/employees/search?name=张三
```

Fence 62

这里 name=张三 会绑定到方法参数。

5.5. @RequestBody

从请求体中取 JSON 数据，并转成 Java 对象。

请求体：

```
1 | {
2 |     "name": "张三",
3 |     "age": 20
4 | }
```

Fence 63

Java 方法:

```
1  @PostMapping
2  public Employee create(@RequestBody Employee emp) {
3      return service.save(emp);
4  }
```

Fence 64

Spring 会自动把 JSON 转成 `Employee` 对象。

十七、Spring MVC 请求完整流程

PPT 中这一页很重要。

流程是：

1. 客户端发请求
2. 请求到达 `DispatcherServlet`
3. `HandlerMapping` 查找处理器
4. `HandlerAdapter` 调用 Controller
5. Controller 返回结果
6. `ViewResolver` 解析视图
7. 渲染响应并返回客户端

1. 1. DispatcherServlet

它是 Spring MVC 的核心。

你可以理解为：

所有请求的大门口。

所有请求先进入 `DispatcherServlet`，再由它分发给具体 Controller。

2. 2. HandlerMapping

负责找方法。

比如请求：

```
1 | GET /user/list
```

Fence 65

它要找：

```
1 | @GetMapping("/user/list")
2 | public String list() {}
```

Fence 66

3. 3. HandlerAdapter

负责真正调用 Controller 方法。

因为不同 Controller 方法写法不同，参数不同，需要适配器帮忙调用。

4. 4. Controller

处理业务请求。

比如：

```
1 @GetMapping("/users")
2 public List<User> list() {
3     return userService.findAll();
4 }
```

Fence 67

5.5. ModelAndView

传统 Spring MVC 会返回模型和视图。

模型是数据。

视图是页面。

比如：

```
1 return new ModelAndView("userList");
```

Fence 68

意思是返回 `userList.jsp` 页面，并带上一些数据。

6.6. ViewResolver

视图解析器。

它负责把逻辑视图名变成真实页面路径。

比如：

```
1 "userList"
```

Fence 69

解析成：

```
1 /WEB-INF/views/userList.jsp
```

Fence 70

7.7. 返回响应

最后 Spring MVC 把 HTML 页面或 JSON 数据返回给浏览器。

十八、请求参数绑定相关注解

PPT 总结了三个：

注解	用途	示例
@RequestParam	获取查询参数	/user?id=1
@PathVariable	获取路径变量	/user/1
@RequestBody	获取 JSON 请求体	{ "name": "张三" }

Table 3

你可以这样记：

- URL 问号后面的，用 @RequestParam
- URL 路径里面的，用 @PathVariable
- JSON 请求体里的，用 @RequestBody

十九、响应处理相关注解

1. 1. @ResponseBody

表示方法返回值不是页面名，而是直接作为响应数据。

```
1  @ResponseBody
2  @GetMapping("/hello")
3  public String hello() {
4      return "hello";
5  }
```

Fence 71

浏览器直接收到：

```
1  hello
```

Fence 72

2. 2. @RestController

等价于：

```
1  @Controller
2  @ResponseBody
```

Fence 73

所以 REST 接口一般直接用它。

二十、全局异常处理

PPT 讲了：

1. `@ControllerAdvice`
2. `@ExceptionHandler`

1. 1. 为什么需要全局异常处理

如果每个 Controller 方法都写 try-catch，会很乱：

```
1  try {  
2      ...  
3  } catch (Exception e) {  
4      ...  
5  }
```

Fence 74

全局异常处理可以统一处理错误。

2. 2. `@ControllerAdvice`

表示这是一个全局异常处理类。

```
1  @ControllerAdvice  
2  public class GlobalExceptionHandler {  
3  }
```

Fence 75

3. 3. `@ExceptionHandler`

指定处理哪种异常。

```
1  @ExceptionHandler(Exception.class)  
2  @ResponseBody  
3  public Result handleException(Exception e) {  
4      return Result.error("系统异常");  
5  }
```

Fence 76

当 Controller 里抛出异常时，会自动进入这里。

这样可以统一返回：

```
1  {  
2      "code": 500,  
3      "message": "系统异常"  
4  }
```

Fence 77

二十一、Spring MVC 拦截器

PPT 讲了 `HandlerInterceptor` 的三个方法：

1. `preHandle`
 2. `postHandle`
 3. `afterCompletion`
-

1. 1. 拦截器是什么

拦截器就是：

在请求进入 Controller 前后插入一些公共逻辑。

常用于：

- 登录检查
 - 权限校验
 - 日志记录
 - 防重复提交
 - 限流
-

2. 2. `preHandle`

Controller 执行前调用。

如果返回 `true`，继续执行。

如果返回 `false`，请求被拦住。

常用于登录判断：

```
1 public boolean preHandle(...) {
2     if (用户已登录) {
3         return true;
4     }
5     return false;
6 }
```

Fence 78

3. 3. `postHandle`

Controller 执行后、视图渲染前调用。

可以修改模型数据。

现在前后端分离项目中用得相对少一些。

4.4. `afterCompletion`

整个请求完成后调用。

无论成功还是异常，一般都会执行。

适合做资源清理、日志收尾等工作。

二十二、常见错误：依赖注入缺失

PPT 提到一个高频错误：

忘记加 `@Autowired` 或 `@Resource`，导致对象为 `null`。

比如：

```
1  @Controller
2  public class UserController {
3      private UserService userService;
4
5      public void test() {
6          userService.findAll();
7      }
8  }
```

Fence 79

这里 `userService` 没有注入，默认是 `null`。

调用时会报：

```
1  NullPointerException
```

Fence 80

也就是空指针异常。

正确写法：

```
1  @Controller
2  public class UserController {
3      @Autowired
4      private UserService userService;
5  }
```

Fence 81

更推荐构造函数注入：

```
1  @Controller
2  public class UserController {
3      private final UserService userService;
4
5      public UserController(UserService userService) {
6          this.userService = userService;
7      }
8  }
```

Fence 82

二十三、综合应用：把 SSM 串起来

PPT 最后一章是综合应用。

它强调你要掌握完整 workflow。

以“查询用户列表”为例：

1. 完整流程

1.1 第一步：前端发请求

```
1 | GET /api/users
```

Fence 83

1.2 第二步：Controller 接收请求

```
1 | @RestController
2 | @RequestMapping("/api/users")
3 | public class UserController {
4 |     @Autowired
5 |     private UserService userService;
6 |
7 |     @GetMapping
8 |     public List<User> list() {
9 |         return userService.findAll();
10 |    }
11 | }
```

Fence 84

1.3 第三步：Service 处理业务

```
1 | @Service
2 | public class UserService {
3 |     @Autowired
4 |     private UserMapper userMapper;
5 |
6 |     public List<User> findAll() {
7 |         return userMapper.findAll();
8 |     }
9 | }
```

Fence 85

1.4 第四步：Mapper 调用 SQL

```
1 @Repository
2 public interface UserMapper {
3     List<User> findAll();
4 }
```

Fence 86

1.5 第五步：XML 写 SQL

```
1 <select id="findAll" resultType="User">
2     SELECT * FROM user
3 </select>
```

Fence 87

1.6 第六步：返回结果

数据库返回数据

- MyBatis 封装成 Java 对象
 - Service 返回给 Controller
 - Controller 返回 JSON 给前端。
-

二十四、代码规范重点

PPT 最后强调了几个工程规范。

1. 1. 组件扫描路径要对

如果你的类在：

```
1 | com.example.service
```

Fence 88

但配置扫描的是：

```
1 | @ComponentScan("com.test")
```

Fence 89

Spring 就扫描不到你的类。

结果就是 Bean 创建失败，注入失败。

2. 2. 分层注解要正确

层	注解
Controller 控制层	@Controller / @RestController
Service 业务层	@Service
DAO / Mapper 持久层	@Repository

Table 4

不要乱用。

3. 3. Mapper 接口方法名要和 XML 的 id 对应

接口：

```
1 | List<User> findAll();
```

Fence 90

XML：

```
1 | <select id="findAll" resultType="User">
2 |     SELECT * FROM user
3 | </select>
```

Fence 91

这里 `findAll` 必须一致。

如果接口叫 `findAll`，XML 写成 `queryAll`，MyBatis 就找不到对应 SQL。

4. 4. 多参数时使用 @Param

例如：

```
1 List<User> findByNameAndAge(@Param("name") String name,  
2                             @Param("age") Integer age);
```

Fence 92

XML 中可以写：

```
1 <select id="findByNameAndAge" resultType="User">  
2     SELECT * FROM user  
3     WHERE name = #{name}  
4     AND age = #{age}  
5 </select>
```

Fence 93

如果不加 @Param，多参数时容易出现参数绑定错误。

二十五、整份 PPT 的核心记忆口诀

你可以这样记：

Spring 管对象，AOP 管增强，MyBatis 管 SQL，Spring MVC 管请求。

再展开：

内容	记忆重点
Spring Core	IoC、DI、Bean、组件扫描
Spring AOP	代理、通知、事务
MyBatis	<code>#{} </code> 、 <code>\${} </code> 、动态 SQL、缓存
Spring MVC	DispatcherServlet、Controller、请求参数、REST
综合应用	Controller → Service → Mapper → XML → DB

Table 5

二十六、考试/复习最容易考的点

你重点背这些：

1. **IoC 是什么？**

对象创建权交给 Spring 容器。

2. **DI 是什么？**

Spring 自动把依赖对象注入进来。

3. **Bean 是什么？**

被 Spring 容器管理的对象。

4. **Spring 默认 Bean 作用域是什么？**

Singleton，单例。

5. **构造函数注入和 Setter 注入区别？**

构造函数注入更安全，Setter 注入更灵活。

6. **AOP 是什么？**

把日志、事务、权限等公共功能从业务代码中抽离。

7. **JDK 动态代理和 CGLIB 区别？**

JDK 要接口，CGLIB 通过继承生成子类。

8. **事务默认什么时候回滚？**

RuntimeException 和 Error。

9. **`#{}` 和 `${}` 区别？**

`#{}` 预编译，安全；`${}` 字符串拼接，有 SQL 注入风险。

10. **动态 SQL 常见标签？**

`<if>`、`<where>`、`<set>`、`<foreach>`、`<choose>`。

11. **Spring MVC 请求流程？**

DispatcherServlet → HandlerMapping → HandlerAdapter → Controller → ViewResolver → Response。

12. **`@RestController` 是什么？**

`@Controller` + `@ResponseBody` ，返回 JSON 数据。

13. **`@RequestParam`、`@PathVariable`、`@RequestBody` 区别？**

查询参数、路径参数、请求体 JSON。

14. **拦截器三个方法？**

`preHandle`、`postHandle`、`afterCompletion`。

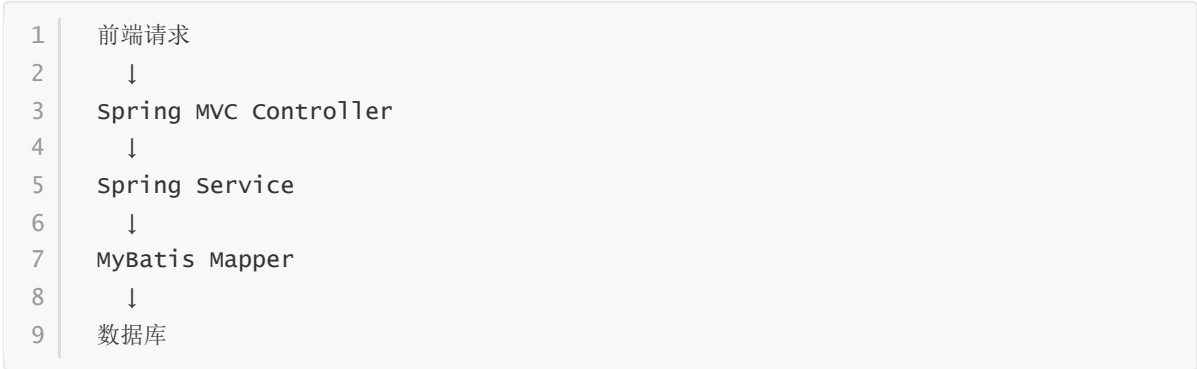
15. **Mapper 接口和 XML 要注意什么？**

方法名和 SQL 标签 id 必须一致。

二十七、你作为小白应该怎么学这份 PPT

建议你按这个顺序理解：

第一步，先理解整体流程：



Fence 94

第二步，再理解 Spring：

- 1 | Spring 帮你创建对象、管理对象、注入对象

Fence 95

第三步，再理解 MyBatis：

- 1 | Mapper 接口调用 XML 里的 SQL，去数据库查数据

Fence 96

第四步，再理解 AOP：

- 1 | 不改业务代码，自动加日志、事务、权限等功能

Fence 97

第五步，最后背注解和常见区别题。

二十八、最终总结

这份 PPT 的核心不是让你死记很多术语，而是让你理解一套 Java 后端项目是怎么跑起来的：

Spring Core 负责创建和管理对象。

Spring AOP 负责给方法自动加事务、日志等公共功能。

MyBatis 负责执行 SQL、操作数据库。

Spring MVC 负责接收请求、调用业务、返回结果。

综合应用 就是把它们串起来，形成一个完整的后端系统。

你只要先抓住这条主线：

1 | 请求进来 → Controller → Service → Mapper → SQL → 数据库 → 返回结果

Fence 98

再慢慢补充每个框架的细节，就能把这份 PPT 学明白。